Description :

Le programme à deux fonction principal : « comparer_algorithmes » et « lancer_algo » le premier sert à comparer plusieur algo sur une liste de graph (avec possibilité de rajouter des graph généré aleatoirement (faudrat modifier ça en procéduralement)) et ça affiche le idleness moyen (minimum, maximum et moyen) et le idleness max (minimum, maximum et moyen)

Le deuxième sert à choisir un algo et le lancer sur un graph (soit du prof soit généré aléatoirement

Les différante classe :

Liaison : représente un lien entre 2 nœud il a une cible et un poids

Nœud : représente un nœud, il a

- un id
- (x, y) les position sur la carte
- temps_non_visite : temps depuis dernière visite
- temps_visites : la liste avec tout les temps d'atente enregistrer quand un agent arrive
- et laison : les liaison avec les autre nœud

Agent : il représente l'agnet, que ce soit un drone ou autre

- id
- noeud_actuel : point où il est
- destination : point où il va
- vitesse : à quel point le poids l'affect
- avancer(dt) : le faire boiger vers det en un temp dt

fonction craph :

choisir_fichier_graphe() : ouvre un fenaitre pour choisir un fichier graph

ouvrir_graphe(chemin) : lit le fichier xml et transphorme le 3d n 2d et créer les objet noeuf et liaison

creer_graphe(nb_noeuds) : fabrique un graphe avec nb_noaud les laiison sont faite selon max_voisin et dist_max_laison

copier_graphe(noeuds_source) : permet de dupliquer un graph qui est utile pour comparer des algo dans les même condition

fonction affichage graph

afficher_liaisons(screen, noeuds) : fonction qui mermet d'afficher les liaison si on l'a activé

afficher_graphe(screen, noeuds, font) : affiche les différen nœud plus le temps passe plus ils passe de vert au rouge en temps_max s

afficher_agents(screen, agents, font) :affiche les agent sous forme de rond bleu

fonction algo

charger_algorithme(chemin, numero) : charge le fichier py pour utiliser la fonction choisir_destination

executer_test(noeuds_base, positions_depart, algo, graine, temps_simulation) : fait tourné la simu en arrière plan pour calucler idleness

afficher_bilan(nom, resultats) : affiche résultat d'un algo sur plusieurs graph, affcihe le idleness moyen (minimum, maximum et moyen) et le idleness max (minimum, maximum et moyen)

comparer_algorithmes() : on selectionne un dossier de graph, on indique combien de grach aléatoire on veut en plus, demande plusieurs .py avec les fonction choisir_destination et il sert à comparer plusieur algo sur la liste de graph

> ps : si la graph est gros la vitesse des agent est égal à 4 fois la moyenne des poids et le temps de suimu est égal au max entre 30 et le nombre de nœud /4
>
> ps 2 : le nombre d'agent est toujours fixé à 10 il faudrat voir comment on fait

lancer_algo() : lance la simulation visuel sur l'algo importer au début de code

construction de l'algo :

voici un code de la fonction minimum (elle est random)

```
3  def choisir_destination(self, agents, TEMPS_SIMULATION):
4
5      # choi au pif du voisin
6      liaison = random.choice(self.noeud_actuel.liaisons)
7
8      self.destination = liaison.cible
9      self.noeud_depart_trajet = self.noeud_actuel
10
11     # calc du temps de trajet
12     self.temps_trajet_total = liaison.poids / self.vitesse
13     self.temps_trajet_ecoule = 0.0
```

Il faut au minimum qu'elle fasse 4 chose :

1) elle doit modifier self.destination vers la destination choisit via « self.destination = liaison.cible » (ligne 8) on peut aussi faire via « self.destination = meilleur_noeud »
2) elle doit dire qu'elle part du nœud où elle est présente via « self.noeud_depart_trajet = self.noeud_actuel » (ligne9)
3) elle doit calucler le temps de trajet via « self.temps_trajet_total = liaison.poids / self.vitesse » ligne (12)
4) elle doit dire quelle vient de commencer son trajet via « self.temps_trajet_ecoule = 0.0 » ligne(13)

variable début :

LARGEUR, HAUTEUR = 1000, 700 #taille de la fenaitre

NB_NOEUDS = 100 # nombre de noeud sur un graphe aleatoire

NB_AGENTS = 10 # nombre d'agent dispo / de dronne

DIST_MAX_LIAISON = 200 # dist max sur la carte pr connecter 2 nœuds sur un graphe aleatoire

MAX_VOISINS = 4 # nb max de voisin par nœud sur un graphe aleatoire

AFFICHER_LIAISONS = True #si on souhaite ou non afficher les liaison

RAYON_NOEUD = 10 #taille visuel noeud

RAYON_AGENT = 8 #taille visuel agent

FPS = 120 #fps (nombre image second)

TEMPS_SIMULATION = 50 # en sec le temps d'execution de la simulation

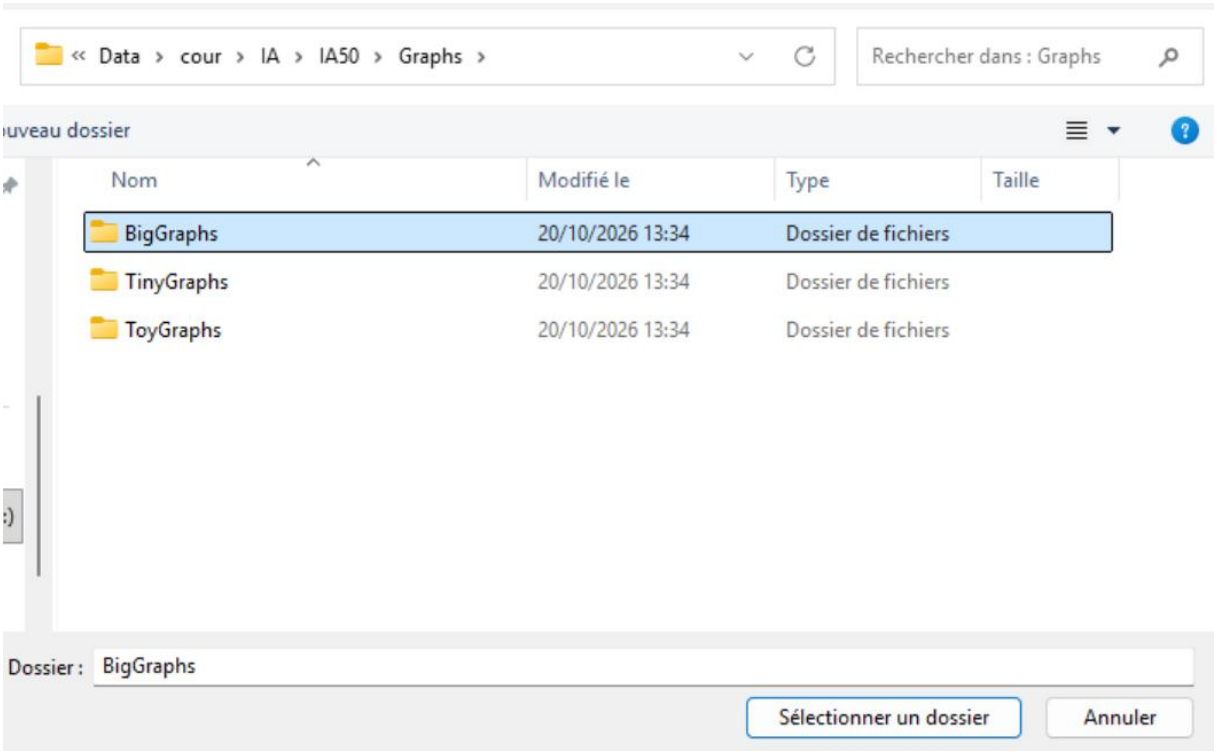
TEMPS_MAX = 10 # temps pr passer du vert au rouge

vitesse_agents = 10 #à quel point les agent vont vite / à quel point ils sont afecter par le poids

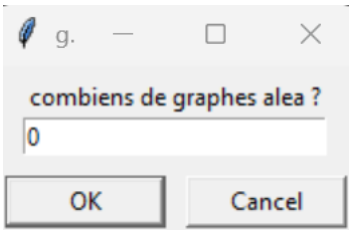
exemple utilisation :

comparer_algorithmes

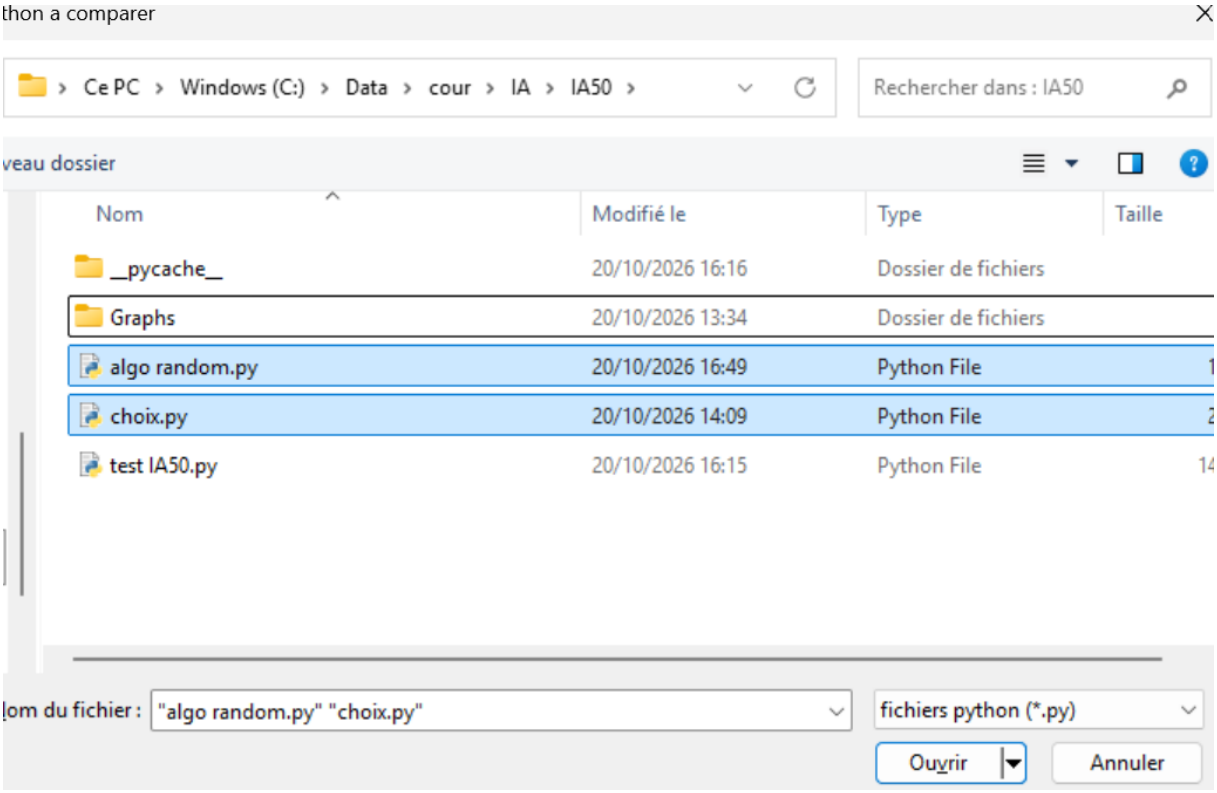
on selectionne un DOSSIER contenant les graph (si on ne veut que des graph aléatoir on peut selectionné un dossier vide ou sans graph)



On dit combien on veut de graph alea en plus



On selectionne les algo qu'on souhaite comparer



Il affiche le idleness moyen et parmis les différent graph il affiche le min, max et la moyenne de même pour le idleness max

```
bilan final

algo : algo random.py
idlenes moy -> min: 0.46s, max: 15.05s, moy: 3.11s
idlenes max -> min: 6.73s, max: 258.29s, moy: 48.08s

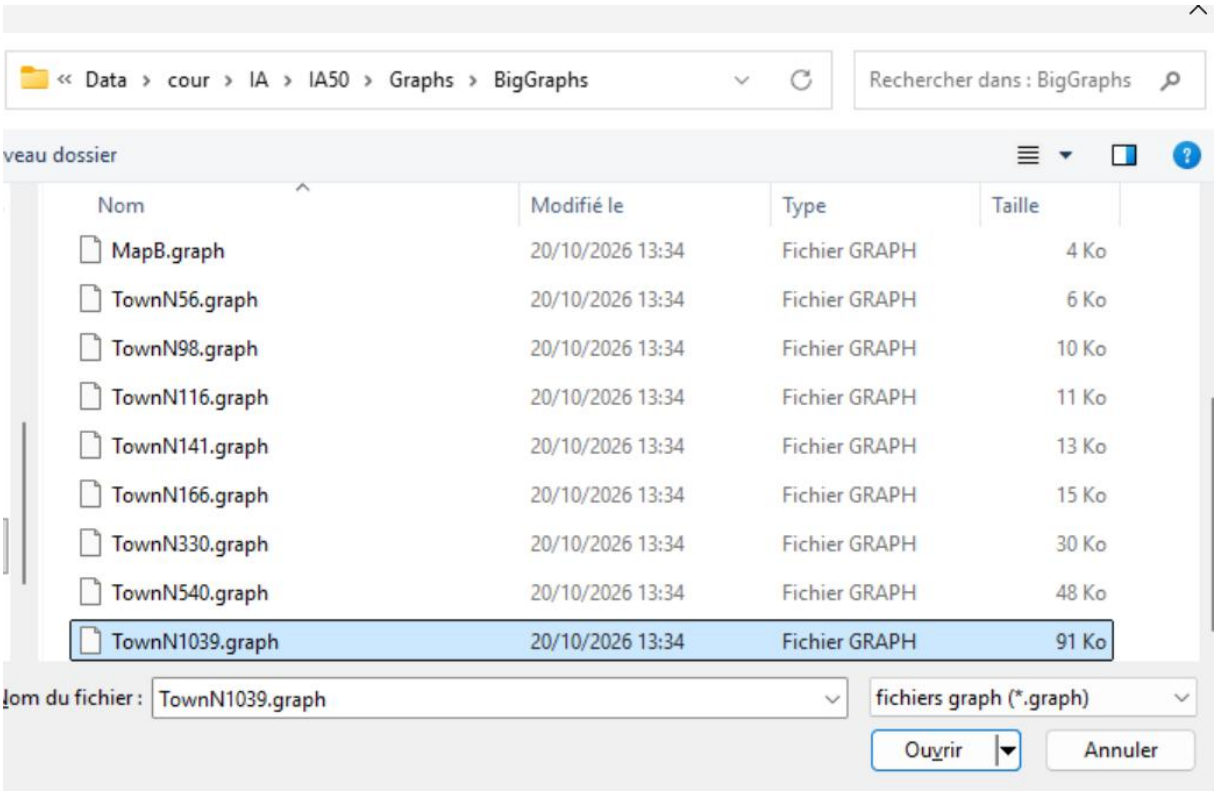
algo : choix.py
idlenes moy -> min: 0.31s, max: 24.05s, moy: 4.24s
idlenes max -> min: 1.06s, max: 146.66s, moy: 20.21s
```

lancer_algo

l'algo utiliser vient de la ligne 11 où choix est le nom du fichier py est choisir dest est le nom de la fonction à l'interieur de ce fichier (ne pas modifier le nom):

```
from choix import choisir_destination
```

il demande de selectionné un graph si on le fait la simu est fait sur celui-ci si on annule alors elle sera faite sur un graphe aléatoire



Ensuite il affcihe le graph avec les agent

